AgentForge

让「AI 说它做完了」变成能被机械证明的交付

一个带确定性验收的多角色 AI 研发引擎

11 条锚点 · 5 个角色 · 8 个阶段 · 401 个测试 · 靶场零误报

实跑结果（可复核）： AI 产出的茶歇系统 —— 单元测试 22 / 22 ，端到端验收 33 / 33 ，零第三方依赖即可启动

产物： demo/coffeebreak/ · 证据： docs/competition/evidence/

刘泽 · B23040610

问题

AI 编程的失败是「假绿灯」式的

看起来完整，实际跑不起来 —— 而工具自己不知道

表面指标（工具说）

- ✓ 18 个模块 / 4,443 行代码
- ✓ tsc --noEmit 通过（编译没问题）
- ✓ 17 个契约端点「代码里全都实现了」
- ✓ 写了 39 条测试

真实结果（跑一下）

- ✗ 12 个端点一律返回 HTTP 501
- ✗ 39 条测试里 26 条失败
- ✗ 业务模块一次都没被调用
- ✗ A7 锚点当时报「全部端点已实现」

根子是一行代码：鉴权通过后直接 sendJson(res, 501, { error: 'not implemented yet' }) 然后 return —— 从不调用业务模块

而 4 个 handleXxx() 全都「导出了、无人调用」。这是我自己跑出来的真实产物，不是编的例子。

根因

不是模型不行，是系统在让角色「盲写」

30 次调用、 113 万 token ，而每次调用的输入恒定在 10.6k



横轴：项目规模 0 → 4,443 行 ｜ 纵轴：输入 token ≈ 10.6k （几乎不动）

95% 24,000 字符的上下文被「需求 + PRD + 任务图」吃掉

4.2% 54 万字符的工件里，真正送达模型的只有这么多

2.2 : 1 输出 / 输入 —— 每次都在从零重写，而不是修改

关键判据

契约（ 60,720 字符）整件丢失 → 后端照 PRD 散文自己发明接口；别人的代码一件没进 → 不可能接线；测试报告一件没进 → 连修 6 轮不收敛。

架构

编排内核零 LLM，判断不可能被模型带偏

入口	CLI · Web 控制台	cli-run.ts · server/
编排层	8 阶段状态机 · 唯一 Gate · 账本 · 机械裁判 · 圆桌 · 逃生	orchestrator/（零 LLM 调用）
验收层	A1–A8 事实锚（编译 / 测试 / 探针 / 契约 / 基准） · B1–B3 语义锚	anchors/（LLM 提议 + 机械核验）
角色层	pm · backend · frontend · test · host	roles/（上下文装配 + schema 门禁）
能力层	Provider 抽象 · 录制回放 · 三层记忆 · 工件库 · 哈希链日志	llm/ · memory/ · core/

三个设计决定

① 编排里没有一次 LLM 调用

② SKIPPED ≠ PASS

③ 8 个阶段共用同一个 Gate

零依赖： Node ≥ 24 原生执行 TypeScript，克隆即跑（401 个测试； CI 断言仓库根目录没有 node_modules）

Agent 链路

五个角色不直接对话，只传接口

谁能看到什么由代码强制，不靠提示词自律

✗ 全文互传

把全部源码堆进一个大上下文 =
把多角色系统退化成单一大上下文，
切分就白做了；
而且下一个更大的项目会再次溢出。

实测： 11 个模块全文 = 20 万字符

✓ 接口卡：只给导出签名

```
export function handlePapersRoute(req: PapersRequest): PapersResponse | null
export function listPapers(): PaperRecord[]
export interface PaperRecord
```

实测： 11 个模块的全部签名 ≈ 5 千字符
与全文相差 40 倍 —— 这是「只传接口」的量化价值

六种通信通道
工件库 · 读权限矩阵 · 契约（唯一必须送达） · 模块接口卡 · 工单 · 圆桌

五层配额（从「谁先谁得」改成「谁需要谁得」）
规格 → 执行事实 → 模块接口 → 代码全文 → 需求计划（关键三件保证送达）

异常处理

四层逃生，机械判定，保证不死锁

decideEscape() 的三个分支穷举了所有情况 —— 它一定返回一个可执行动作

① 打回返工

能归因就先打回；一条都归不了因才开会

便宜、确定、有明确验收条件

② 圆桌会议

唯一允许 LLM 协商的地方（T1–T5）

只「归因完全失灵」才触发

③ 升级真人

打包成仲裁包，人只需做一次决定

圆桌无有效决议时

④ 带债通过

写 DebtRecord + 需求标 accepted_with_debt

真人不可用时的最终出口

四个防空转机制

阻断 3 次后权限自动终止
否则两个 Agent 能互相否决到死

签名未变 → 不再开会
实测过某阶段连开 6 次圆桌

修复无效 → 直接转逃生
不重复付费

上下文超长 → 预算减半重试
不让一次 400 打死整轮 run

真实日志

[CONTRACTING] 工单修复后硬失败签名未变（pm 的修复没有改变任何东西），不再重复重试，转逃生流程 → 带债通过（受影响需求 3 条）

可观测性

能回答「它到底看到了什么」

在这一行存在之前，「为什么它老是做不对」只能靠猜 —— 我们为此花了两小时复算

run 日志里的输入审计（每次调用都记）

上下文： 23,177 / 60,000 字符（ 39% ） 注入 3 个 · 丢弃 0 个
其中契约摘要 1 个
未注入： TaskGraph-001(15520·tier-cap)

injected[]: {id, kind, chars, digest? | card?}
omitted[]: {id, kind, chars, reason: tier-cap | no-space}

哈希链日志

任何一条被改，链就断

回放

同一批模型输出可重放，把不确定性从实验里摘出去

锚点靶场

45 个样本标定检查本身准不准

靶场： 45 个样本（ 30 注入 + 15 干净对照）

TP 30　FN 0　FP 0　TN 15　严重度不符 0

「干净通过率」比「检出率」更重要：一条会对好代码报警的检查，会让角色学会糊弄它 —— 此后所有真报都会被降权。

结论

已经做到的，和还没做到的

已经做到

一个 AI 写出来、真的能跑的系统
单元测试 22 / 22，端到端 33 / 33，零依赖启动

11 条确定性锚点 + 401 个测试守着「检查本身是对的」
靶场 45 样本零误报

一条可复用的方法论
先让失败可见，再谈减少失败

还没做到（诚实）

没有一次自主跑到最终交付
成功那轮在最后一道关卡之后进程被环境回收 —— 产品是好的，编排没结账

四轮并行只成一，方差还大
卡点集中在契约生成质量

B1 / B3 在成功那轮是 SKIPPED
SKIPPED ≠ PASS，所以我们不算它通过

下面是它跑出来的真实系统，以及它在生成过程中自己报过的两个问题。